

LAB # 02

Implementation Of Activation Function

Lab Tasks:

1. Implement a step function that outputs 1 if input ≥ 0 , else 0.
 - Input Values: [-3.0, -1.0, 0.0, 2.0, 5.0]
 - Expected Output: [0, 0, 1, 1, 1]
 - Visualization: Plot the step function for inputs between -5 to 5.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt

# --- Step Function Implementation ---
def step_function(x):
    """
    Step activation function:
    Returns 1 if x >= 0, else 0
    """
    return np.where(x >= 0, 1, 0)

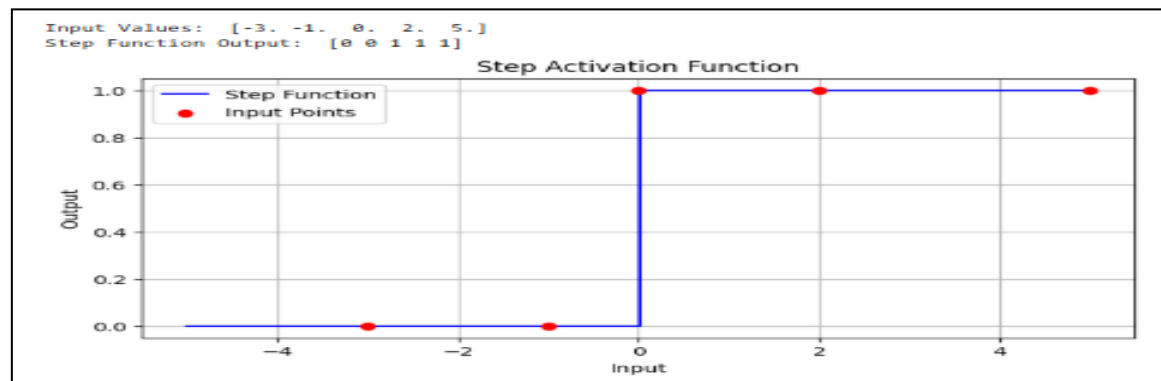
# --- Input Values ---
inputs = np.array([-3.0, -1.0, 0.0, 2.0, 5.0])
outputs = step_function(inputs)

print("Input Values: ", inputs)
print("Step Function Output: ", outputs)

# --- Visualization ---
x_vals = np.linspace(-5, 5, 500)
y_vals = step_function(x_vals)

plt.figure(figsize=(8, 4))
plt.step(x_vals, y_vals, where='post', label='Step Function', color='blue')
plt.scatter(inputs, outputs, color='red', zorder=5, label='Input Points')
plt.title('Step Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
plt.show()
```

Output:-



2. Implement ReLU: $f(x) = \max(0, x)$.

- Input Values: [-4.0, -2.0, 0.0, 3.0, 5.0]
- Expected Output: [0, 0, 0, 3, 5]
- Visualization: Plot the ReLU graph between -5 and 5

Code:-

```
import numpy as np
import matplotlib.pyplot as plt

# --- ReLU Function Implementation ---
def relu(x):
    """
    ReLU activation function:
    Returns max(0, x) for each input element
    """
    return np.maximum(0, x)

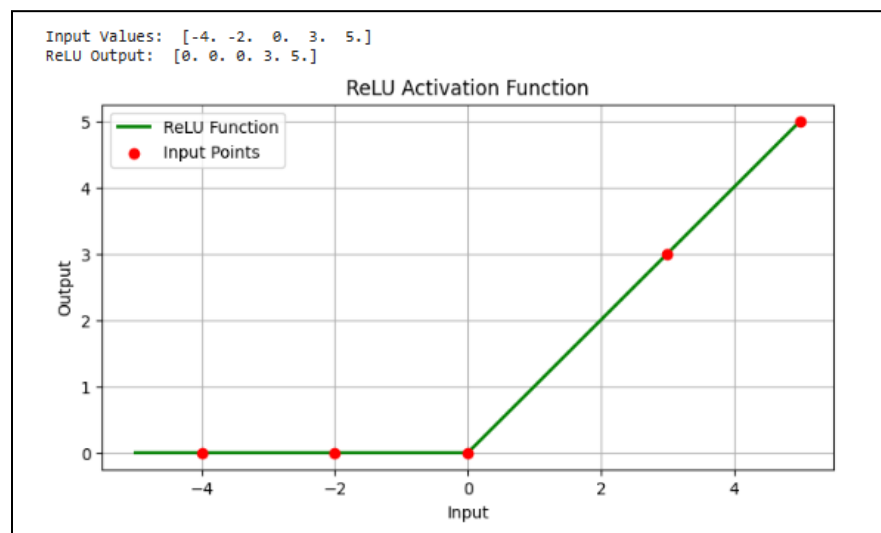
# --- Input Values ---
inputs = np.array([-4.0, -2.0, 0.0, 3.0, 5.0])
outputs = relu(inputs)

print("Input Values: ", inputs)
print("ReLU Output: ", outputs)

# --- Visualization ---
x_vals = np.linspace(-5, 5, 500)
y_vals = relu(x_vals)

plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_vals, label='ReLU Function', color='green', linewidth=2)
plt.scatter(inputs, outputs, color='red', zorder=5, label='Input Points')
plt.title('ReLU Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
plt.show()
```

Output:-



3. Implement Sigmoid: $f(x) = 1 / (1 + e^{-x})$.

- Input Values: [-2.0, 0.0, 1.0, 3.0]
- Expected Output: $\approx$ [0.119, 0.5, 0.731, 0.953]
- Visualization: Plot the sigmoid curve and explain why sigmoid compresses values between 0 & 1.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt

# --- Sigmoid Function Implementation ---
def sigmoid(x):
    """
    Sigmoid activation function:
     $f(x) = 1 / (1 + e^{-x})$ 
    """
    return 1 / (1 + np.exp(-x))

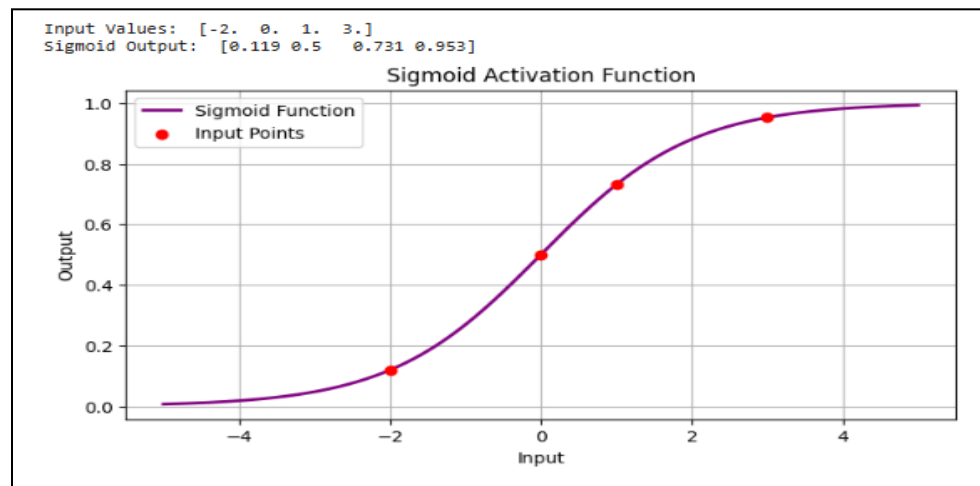
# --- Input Values ---
inputs = np.array([-2.0, 0.0, 1.0, 3.0])
outputs = sigmoid(inputs)

print("Input Values: ", inputs)
print("Sigmoid Output: ", np.round(outputs, 3))

# --- Visualization ---
x_vals = np.linspace(-5, 5, 500)
y_vals = sigmoid(x_vals)

plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_vals, label='Sigmoid Function', color='purple', linewidth=2)
plt.scatter(inputs, outputs, color='red', zorder=5, label='Input Points')
plt.title('Sigmoid Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
plt.show()
```

Output:-



4. Implement Tanh: $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$.

- Input Values: [-1.5, 0.0, 1.0, 2.0]
- Expected Output (approx): [-0.905, 0.0, 0.762, 0.964]
- Visualization: Plot Tanh and compare it with Sigmoid.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt

# --- Tanh Function Implementation ---
def tanh(x):
    """
    Tanh activation function:
    f(x) = (e^x - e^(-x)) / (e^x + e^(-x))
    """
    return np.tanh(x)

# --- Sigmoid Function for Comparison ---
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

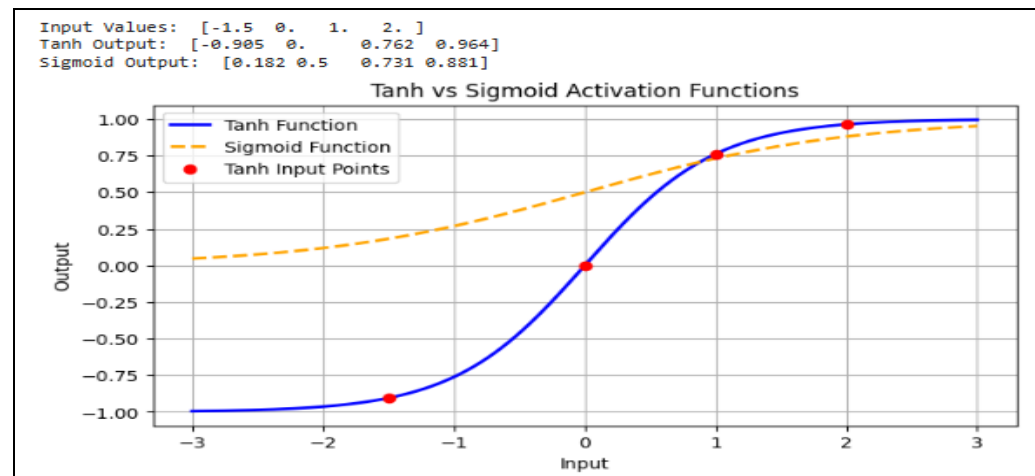
# --- Input Values ---
inputs = np.array([-1.5, 0.0, 1.0, 2.0])
tanh_outputs = tanh(inputs)
sigmoid_outputs = sigmoid(inputs)

print("Input Values: ", inputs)
print("Tanh Output: ", np.round(tanh_outputs, 3))
print("Sigmoid Output: ", np.round(sigmoid_outputs, 3))

# --- Visualization ---
x_vals = np.linspace(-3, 3, 500)
tanh_vals = tanh(x_vals)
sigmoid_vals = sigmoid(x_vals)

plt.figure(figsize=(8, 4))
plt.plot(x_vals, tanh_vals, label='Tanh Function', color='blue', linewidth=2)
plt.plot(x_vals, sigmoid_vals, label='Sigmoid Function', color='orange', linewidth=2, linestyle='--')
plt.scatter(inputs, tanh_outputs, color='red', zorder=5, label='Tanh Input Points')
plt.title('Tanh vs Sigmoid Activation Functions')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
plt.show()
```

Output:-



5. Implement Leaky ReLU: $f(x) = x$ if $x \geq 0$ else $\alpha * x$.

- Input Values: [-3.0, -1.0, 0.0, 2.0]
- Test with $\alpha = 0.1$: Expected Output $\rightarrow$ [-0.3, -0.1, 0.0, 2.0]
- Visualization: Plot Leaky ReLU for $\alpha = 0.1$ and $\alpha = 0.3$.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt

# --- Leaky ReLU Function Implementation ---
def leaky_relu(x, alpha=0.1):
    """
    Leaky ReLU activation function:
    f(x) = x if x >= 0 else alpha * x
    """
    return np.where(x >= 0, x, alpha * x)

# --- Input Values ---
inputs = np.array([-3.0, -1.0, 0.0, 2.0])
alpha1 = 0.1
alpha2 = 0.3

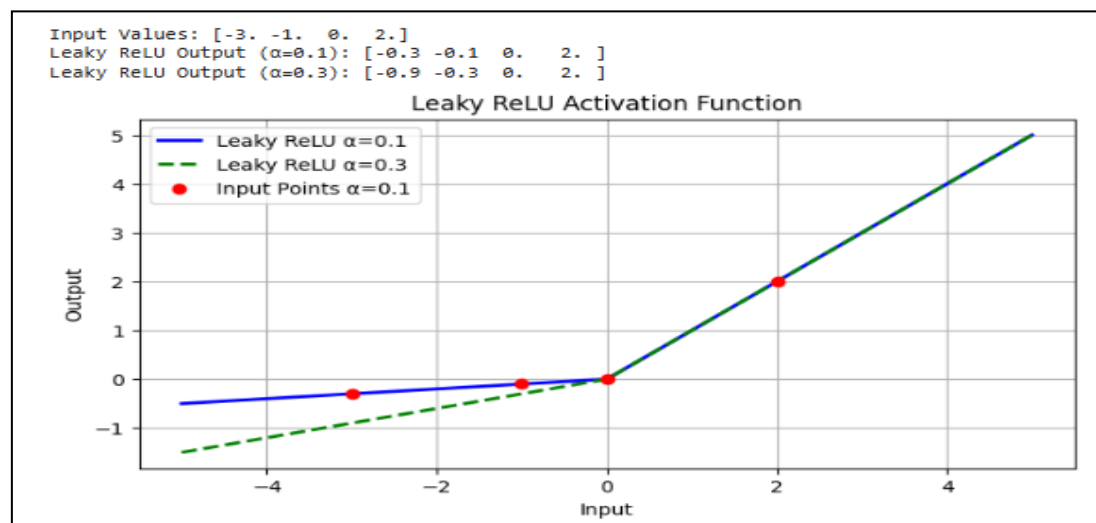
outputs_alpha1 = leaky_relu(inputs, alpha=alpha1)
outputs_alpha2 = leaky_relu(inputs, alpha=alpha2)

print(f"Input Values: {inputs}")
print(f"Leaky ReLU Output (α={alpha1}): {outputs_alpha1}")
print(f"Leaky ReLU Output (α={alpha2}): {outputs_alpha2}")

# --- Visualization ---
x_vals = np.linspace(-5, 5, 500)
y_alpha1 = leaky_relu(x_vals, alpha=alpha1)
y_alpha2 = leaky_relu(x_vals, alpha=alpha2)

plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_alpha1, label='Leaky ReLU α=0.1', color='blue', linewidth=2)
plt.plot(x_vals, y_alpha2, label='Leaky ReLU α=0.3', color='green', linewidth=2, linestyle='--')
plt.scatter(inputs, outputs_alpha1, color='red', zorder=5, label='Input Points α=0.1')
plt.title('Leaky ReLU Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.legend()
plt.show()
```

Output:-



6. Implement Softmax to convert inputs into probabilities.

- Input Values: [1.0, 2.0, 3.0]
- Expected Output (approx): [0.090, 0.245, 0.665]
- Explanation: Why the output values always sum to 1.

Code:-

```
import numpy as np

# --- Softmax Function Implementation ---
def softmax(x):
    """
    Softmax activation function:
    Converts a vector of values into probabilities that sum to 1
    """
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)

# --- Input Values ---
inputs = np.array([1.0, 2.0, 3.0])
outputs = softmax(inputs)

print("Input Values: ", inputs)
print("Softmax Output: ", np.round(outputs, 3))
print("Sum of Softmax Outputs: ", np.sum(outputs))
```

Output:-

```
Input Values: [1. 2. 3.]
Softmax Output: [0.09 0.245 0.665]
Sum of Softmax Outputs: 0.9999999999999999
```

Bonus Task: Plot the following activation functions: Sigmoid, Tanh, ReLU, and Leaky ReLU on the same graph using an input range of -10 to 10. Use different colors and a legend to clearly identify each function. After plotting, create a short table showing the output range and main problem of each activation function. The task focuses on generating the graph and table only.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Input range
x = np.linspace(-10, 10, 500)

# Activation functions
activations = {
    "Sigmoid": 1 / (1 + np.exp(-x)),
    "Tanh": np.tanh(x),
    "ReLU": np.maximum(0, x),
    "Leaky ReLU": np.where(x >= 0, x, 0.1*x)
}

# Plot all activations
plt.figure(figsize=(10,6))
colors = ['purple', 'blue', 'green', 'orange']
for (name, y), color in zip(activations.items(), colors):
    plt.plot(x, y, label=name, color=color, linewidth=2, linestyle='--' if name=="Leaky ReLU" else '-')
plt.title("Activation Functions Comparison")
plt.xlabel("Input")
plt.ylabel("Output")
plt.grid(True)
plt.legend()
plt.show()

# Quick summary table
table = pd.DataFrame({
    "Activation": ["Sigmoid", "Tanh", "ReLU", "Leaky ReLU"],
    "Output Range": ["0 to 1", "-1 to 1", "0 to ∞", "-∞ to ∞ (small slope)"],
    "Main Problem": [
        "Vanishing gradient",
        "Vanishing gradient",
        "Dying ReLU",
        "Small negative slope avoids dying ReLU"
    ]
})
print(table)
```

Output:-

